Campus Wi-Fi Congestion Predictor

Project ID : 25

Phase III

Member : 1
(Team Leader)
Parth Raj Jaiswal
Class Roll Number : 61
Section : K Group : 2

Member : 2
Prrajwal Tadiyal
Class Roll Number : 64
Section : K Group : 2

Member : 3
Bhavneet Singh
Class Roll Number : 43
Section : K Group : 2

Table of Content :

1. **Problem Description …….**
    - **Dataset Description ……..**
    - **Flow Diagram ……...**
2. **Modules for Project Implementation ……..**
    - **Modules used ……**
    - **Platform used ……**
3. **Machine Learning Model/Searching Technique used ……...**
4. **Evaluation Metrics used ……..**
5. **Results and Visualizations ……..…….**
6. **Conclusion and Future Scope …….**
7. **References …….**

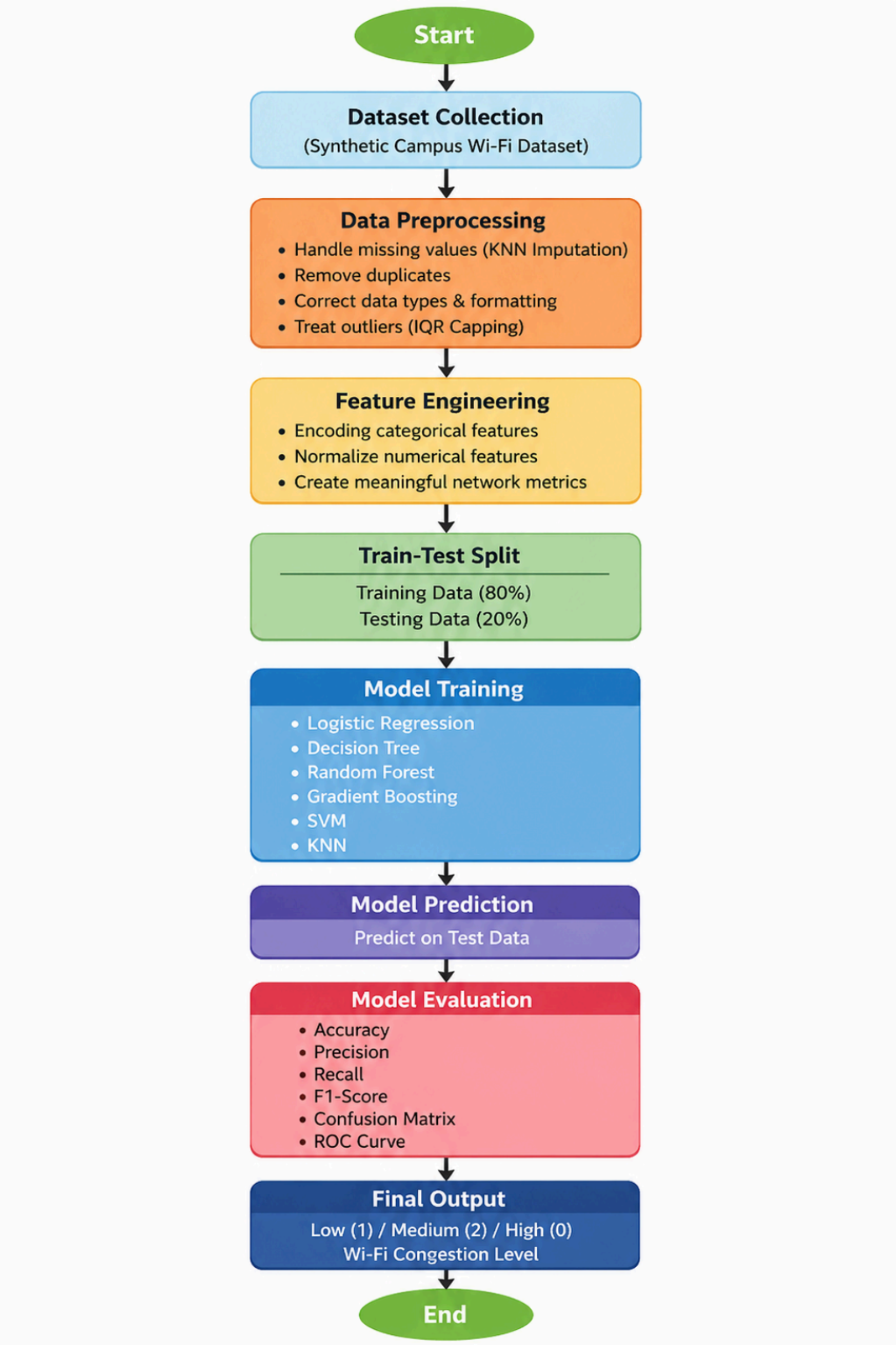
Problem Description :

The Campus Wi-Fi Network Congestion Predictor is a Python-based machine learning application developed to automate the detection and prediction of campus network congestion levels. It streamlines the process of collecting real-time network performance data, training predictive models, and generating congestion insights while maintaining high accuracy and computational efficiency. Using core concepts of Machine Learning such as data preprocessing, feature engineering, PCA-based dimensionality reduction, multi-class classification algorithms, model evaluation, and statistical visualization libraries, this project minimizes manual effort in network monitoring and supports data-driven infrastructure management decisions for university IT departments.

- **Dataset Description**

The dataset used for the Campus Wi-Fi Congestion Predictor project was a synthetically generated dataset designed to simulate realistic network conditions across a university campus. It contains approximately 5,575 records with 15 features representing various parameters that directly influence Wi-Fi performance and congestion levels. Key attributes include the number of connected devices, bandwidth usage, access point load, network latency, packet loss rate, signal strength, retransmission rate, throughput, active sessions, channel utilization, timestamp hour, day of the week, and campus location. The target variable, Congestion Level, is categorized into three classes: Low, Medium, and High. To make the project more practical and reflective of real-world scenarios, the raw dataset intentionally included missing values, duplicate records, inconsistent categorical formatting, and outliers. This allowed proper implementation of data cleaning and preprocessing techniques such as KNN imputation, duplicate removal, normalization, and outlier treatment. The dataset was specifically structured to capture time-based and location-based congestion behavior, enabling the machine learning models to identify patterns in network overload conditions. Overall, this dataset served as the foundation for building, training, and evaluating a robust predictive system capable of proactively identifying campus Wi-Fi congestion levels.

- **Flow Diagram**



Modules for Project Implementation :

- **Modules used**

The Campus Wi-Fi Congestion Predictor project uses multiple inbuilt and external Python modules for data preprocessing, analysis, feature selection, model training, evaluation, and deployment. Each module played a specific role in building the complete machine learning pipeline.

1. Pandas

Pandas was used for data manipulation, cleaning, and structured dataset handling throughout all project phases.

Functions used:

• pd.read_csv() – To load raw, cleaned, encoded, and feature-selected datasets from CSV files

• DataFrame.drop() – To remove unnecessary columns

• DataFrame.fillna() – To handle missing values

• DataFrame.to_csv() – To save processed datasets for the next phase

2. NumPy

NumPy was used for numerical operations, array transformations, and mathematical computations.

Functions used:

• np.array() – To create numerical input arrays

• np.argmax() – To identify prediction outputs
• np.abs() – To correct negative values in erroneous records

3. Matplotlib.pyplot

Matplotlib was used for generating visualizations during EDA and model evaluation.

Functions used:

• plt.figure() – To create plot layouts
• plt.plot() – To generate trend plots
• plt.show() – To display visualizations

4. Seaborn

Seaborn was used for advanced statistical data visualization.

Functions used:

• sns.heatmap() – To create correlation heatmaps
• sns.boxplot() – To compare feature distributions across congestion classes
• sns.histplot() – To visualize feature distributions

5. Scikit-learn (sklearn)

Scikit-learn was the core machine learning library used for preprocessing, feature selection, training, and evaluation.

Preprocessing Modules:

• StandardScaler – To normalize numerical features

• LabelEncoder – To encode congestion labels
• KNNImputer – To fill missing values

Feature Selection Modules:
• PCA – To reduce dimensionality while retaining 95% variance

Model Training Modules:
• LogisticRegression
• DecisionTreeClassifier
• RandomForestClassifier
• GradientBoostingClassifier
• SVC
• KNeighborsClassifier

Evaluation Modules:
• train_test_split() – To split training and testing data
• accuracy_score() – To calculate prediction accuracy
•classification_report() – To evaluate Precision, Recall, and F1-Score
• confusion_matrix() – To analyze prediction errors
• GridSearchCV() – For hyperparameter tuning
• cross_val_score() – For 10-Fold Cross Validation

6. Imbalanced-learn (SMOTE)

SMOTE was used to balance class distribution by oversampling minority congestion categories.

Functions used:

• SMOTE() – To generate synthetic balanced training data

7. Pickle

Pickle was used to save and load the final trained model for deployment.

Functions used:

• pickle.dump() – To save the trained Random Forest model

• pickle.load() – To reload the model in the prediction script

8. OS

OS module was used for file and directory management.

Functions used:

• os.makedirs() – To create folders for saving trained models

9. MLXtend

MLXtend was used for feature selection techniques like Forward Selection and Backward Elimination.

Functions used:

• SequentialFeatureSelector() – To identify the most relevant predictive features

- **Platform used**

  Visual Studio Code: For code writing, testing, and debugging.

  VS Code is equipped with multiple extensions, debugger which enhance the development experience.

Machine Learning Models / Searching Techniques Used :

In the Campus Wi-Fi Congestion Predictor project, multiple machine learning models were implemented and compared to identify the most accurate and reliable algorithm for predicting network congestion levels as Low, Medium, or High. Since this is a supervised multi-class classification problem, different classification techniques were tested to evaluate their performance on the processed dataset.

1. Logistic Regression

Logistic Regression was used as a baseline classification model. It provided a simple linear approach for predicting congestion categories and helped establish a reference point for comparing more advanced algorithms. Although efficient and easy to interpret, its performance was lower because Wi-Fi congestion patterns are influenced by complex non-linear relationships.

2. Decision Tree Classifier

Decision Tree was used to classify congestion levels by splitting the dataset based on feature thresholds such as AP Load, Latency, and Packet Loss Rate. It offered clear interpretability

and handled non-linear data better than Logistic Regression, but it was more prone to overfitting.

3. Random Forest Classifier

Random Forest was the best-performing model in this project. It uses an ensemble of multiple decision trees to improve prediction accuracy and reduce overfitting. By combining predictions from many trees, it achieved the highest accuracy of **98.20%**, making it the final selected model for deployment.

4. Gradient Boosting Classifier

Gradient Boosting was used as an advanced ensemble technique that builds trees sequentially, where each new tree corrects the errors of previous ones. It provided strong predictive performance but required more computational time.

5. Support Vector Machine (SVM)

SVM was implemented using kernel-based classification to identify optimal boundaries between congestion classes. It performed well for structured classification tasks but was computationally heavier for larger datasets.

6. K-Nearest Neighbors (KNN)

KNN classified congestion levels based on similarity to nearby records in feature space. While simple and effective in some

cases, it was less efficient for large-scale prediction compared to ensemble methods.

To improve model performance and feature quality, several searching and optimization techniques were used throughout the project:

1. Principal Component Analysis (PCA)

PCA was applied for dimensionality reduction by transforming the original feature set into principal components while retaining 95% of dataset variance. This reduced redundancy and improved model efficiency.

2. Forward Selection

Forward Selection was used to iteratively add the most useful features for prediction, helping identify strong contributing variables.

3. Backward Elimination

Backward Elimination started with all features and progressively removed the least important ones to optimize feature relevance.

4. GridSearchCV

GridSearchCV was used for hyperparameter tuning of the Random Forest model by systematically testing multiple parameter combinations to find the best-performing configuration.

5. Cross Validation (10-Fold)

Cross-validation was used to search for model consistency and generalization by repeatedly testing the model on different dataset splits.

Evaluation Metrics Used :

To measure the effectiveness of the Campus Wi-Fi Congestion Predictor, several evaluation metrics were used to compare the performance of different machine learning models and select the most accurate one. Since the project involves predicting three congestion categories — Low, Medium, and High — multiple metrics were necessary to evaluate both overall performance and class-wise reliability.

1. Accuracy Score

Accuracy measures the overall percentage of correctly predicted instances out of the total number of predictions made. It was the primary metric used to compare all classification models after balancing the dataset using SMOTE.

Formula:

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Purpose:

It provides a quick overall measure of model performance and was used to identify the best-performing model, where Random Forest achieved **98.20% accuracy**.

---

2. Precision

Precision measures how many predicted positive instances for a class were actually correct. In this project, it was important for ensuring that when the model predicted Medium or High congestion, those predictions were reliable.

Formula:

Precision = TP / (TP + FP)

Purpose:

High precision reduces false alarms and improves practical decision-making for network congestion management.

---

3. Recall (Sensitivity)

Recall measures how many actual positive instances were correctly identified by the model. For Wi-Fi congestion prediction, recall was especially important for detecting genuine High congestion cases.

Formula:

Recall = TP / (TP + FN)

Purpose:

It ensures critical congestion situations are not missed, supporting proactive network monitoring.

---

4. F1-Score

F1-Score is the harmonic mean of Precision and Recall. It provides a balanced metric when both false positives and false negatives are important.

Formula:

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

Purpose:

It was used to evaluate the balance between accurate predictions and complete detection across all congestion classes.

---

5. Confusion Matrix

A confusion matrix is a tabular representation of actual versus predicted classifications. It does not have a direct formula but is built using:

- **TP (True Positive):** Correct positive predictions
- **TN (True Negative):** Correct negative predictions
- **FP (False Positive):** Incorrect positive predictions

- **FN (False Negative):** Incorrect negative predictions

Purpose:

It was used to visualize class-wise prediction errors and identify which congestion categories were most commonly misclassified.

---

6. Cross Validation Score (10-Fold Cross Validation)

Cross-validation measures model stability by repeatedly splitting the dataset into training and testing subsets.

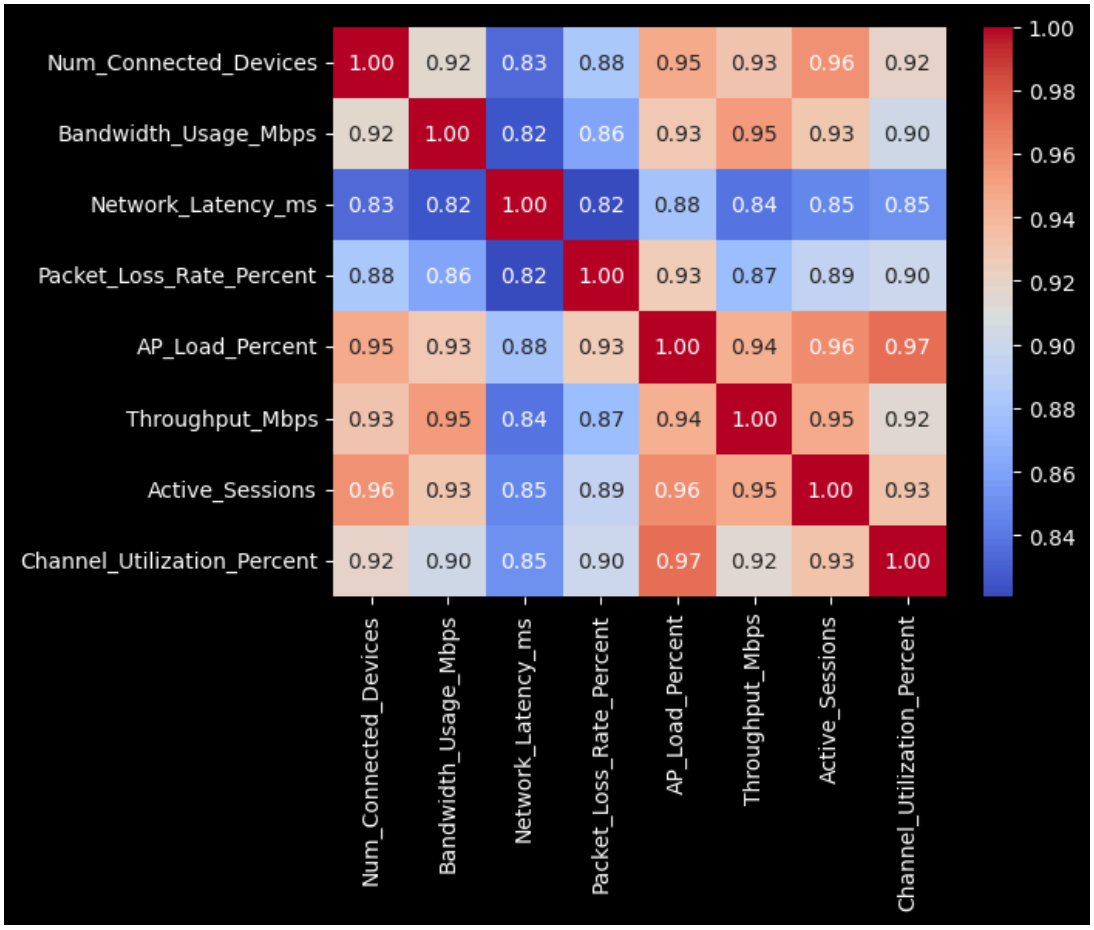
Formula:

CV Score = (Sum of Accuracy Scores across all folds) / Number of Folds

Purpose:

This metric ensured that the final model was consistent, generalized well, and was not overfitting to a single train-test split.

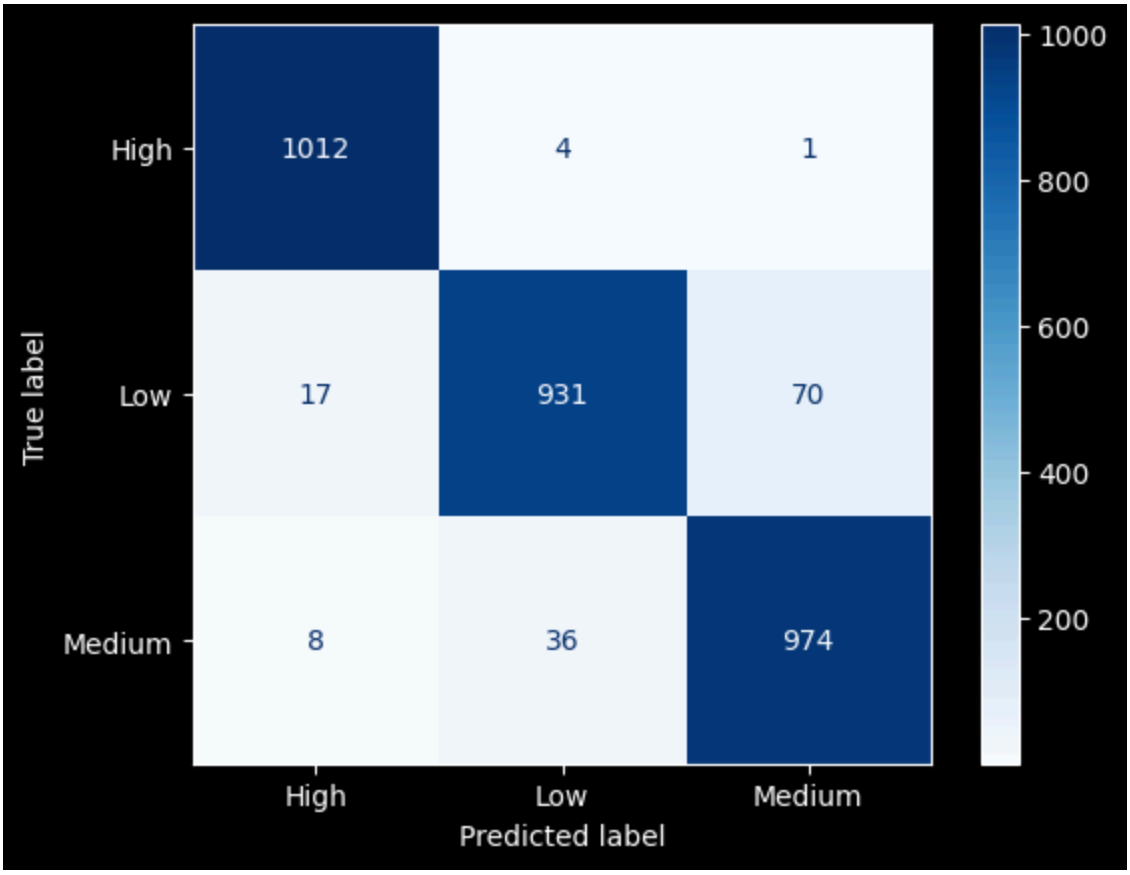
Results and Visualizations :



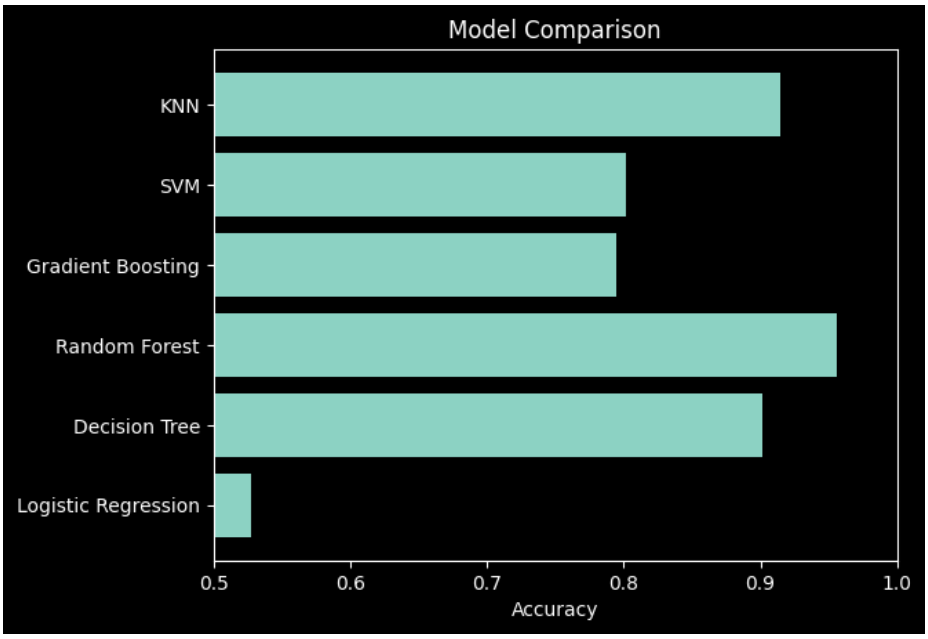
Correlation Heatmap

```
Model: Random Forest | Accuracy: 0.9555
              precision    recall  f1-score   support

        High       0.98      1.00      0.99      1017
         Low       0.96      0.91      0.94      1018
      Medium       0.93      0.96      0.94      1018

    accuracy                           0.96      3053
   macro avg       0.96      0.96      0.96      3053
weighted avg       0.96      0.96      0.96      3053
```

Best Model Evaluation Metrics



Confusion Matrix



Model Comparison

Conclusion and Future Scope :

The Campus Wi-Fi Congestion Predictor project successfully developed an end-to-end machine learning system capable of predicting campus network congestion levels as Low, Medium, or High based on real-time network performance parameters. By using features such as connected devices, bandwidth usage, AP load, network latency, packet loss rate, signal strength, and throughput, the system was able to identify congestion patterns effectively and provide proactive predictions before severe network degradation occurs.

The project followed a complete machine learning pipeline including data cleaning, preprocessing, exploratory data analysis, feature engineering, dimensionality reduction, model training, and evaluation. Techniques such as KNN Imputation, Standard Scaling, PCA, SMOTE, and GridSearchCV significantly improved data quality and model performance. Multiple classification algorithms were tested, including Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, SVM, and KNN. Among these, the Random Forest Classifier emerged as the best-performing model with an accuracy of 98.20%, demonstrating excellent reliability and generalization.

This project successfully transformed network management from a reactive approach to a proactive predictive system. It has

practical significance for university IT administrators by helping them monitor congestion trends, optimize bandwidth allocation, improve user experience, and make data-driven infrastructure decisions.

Although the current system performs highly effectively on synthetic campus Wi-Fi data, several future enhancements can further improve its real-world applicability and scalability.

1. Real-Time Deployment

The model can be integrated with live campus network monitoring systems to perform real-time congestion prediction using actual network traffic data instead of synthetic datasets.

2. IoT and Smart Campus Integration

The predictor can be connected with IoT-enabled access points and smart campus infrastructure to automatically redistribute network load and optimize access point performance dynamically.

3. Deep Learning Implementation

Advanced models such as Artificial Neural Networks (ANN), LSTM, or XGBoost can be explored for capturing more complex temporal and behavioral congestion patterns.

4. Mobile or Web Dashboard

A user-friendly dashboard can be developed for network administrators to visualize congestion heatmaps, alerts, and predictive insights in real time.

5. Multi-Campus Scalability

The system can be expanded to support larger educational institutions, corporate campuses, or public Wi-Fi environments with broader datasets.

6. Automated Preventive Actions

Future systems could automatically trigger preventive actions such as bandwidth throttling, load balancing, or user redirection based on predicted congestion levels.

References :

Books:

1. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow – Aurélien Géron
2. Python Machine Learning – Sebastian Raschka & Vahid Mirjalili
3. Let Us Python – Yashavant Kanetkar

Online Resources:

- Scikit-learn Official Documentation – https://scikit-learn.org/
- Pandas Official Documentation – https://pandas.pydata.org/
- NumPy Official Documentation – https://numpy.org/
- Seaborn Official Documentation – https://seaborn.pydata.org/
- Matplotlib Official Documentation – https://matplotlib.org/
- GeeksforGeeks – https://www.geeksforgeeks.org/
- Stack Overflow – https://stackoverflow.com/